

Indexing and reconstruction

Objective. Map $I \in \mathbb{R}^{B \times C \times H \times W}$ to $F \in \mathbb{R}^{B \times (CHW)}$ and reconstruct $\hat{I}$. The batch index is preserved; width varies fastest, followed by height and channel. This linear ordering supports tensor buffering and accelerator data transfers.

1. Forward mapping

```
F = allocate(B, C*H*W, same dtype as I)
for b in 0..B-1, c in 0..C-1:
    for h in 0..H-1, w in 0..W-1:
        k = (c*H + h)*W + w
        F[b,k] = I[b,c,h,w]
```

2. Reverse mapping

```
I_hat = allocate(B, C, H, W, same dtype as F)
for b in 0..B-1, k in 0..C*H*W-1:
    c = k // (H*W)
    r = k % (H*W)
    h = r // W; w = r % W
    I_hat[b,c,h,w] = F[b,k]
```

Euclidean division gives a unique decomposition $k = c(HW) + hW + w$, where $0 \leq h < H$ and $0 \leq w < W$. The mapping is therefore bijective. Both conversions take $O(BCHW)$ time and allocate $O(BCHW)$ output storage. The reverse conversion requires the original C, H, W as metadata.

3. Manual example: $B = 1, C = 2, H = 2, W = 3$

```
I[0,0,:,:] = [[0,1,2], [3,4,5]]
I[0,1,:,:] = [[6,7,8], [9,10,11]]
F[0,:] = [0,1,2,3,4,5,6,7,8,9,10,11]
```

Source (b, c, h, w)	Forward calculation	Destination (b, k)
$(0, 0, 0, 2)$	$(0 \cdot 2 + 0) \cdot 3 + 2 = 2$	$(0, 2)$
$(0, 0, 1, 0)$	$(0 \cdot 2 + 1) \cdot 3 + 0 = 3$	$(0, 3)$
$(0, 1, 0, 0)$	$(1 \cdot 2 + 0) \cdot 3 + 0 = 6$	$(0, 6)$
$(0, 1, 1, 2)$	$(1 \cdot 2 + 1) \cdot 3 + 2 = 11$	$(0, 11)$

Inverse example. For $k = 10$, $c = \lfloor 10/6 \rfloor = 1$, $r = 10 \bmod 6 = 4$, $h = \lfloor 4/3 \rfloor = 1$, and $w = 4 \bmod 3 = 1$. Thus $F[0, 10]$ returns to $\hat{I}[0, 1, 1, 1] = 10$.

Experiments and measured errors

Every run checks all flattened elements, reconstructed elements, and inverse indices. The signed difference tensor $E = I - \hat{I}$ is computed explicitly and saved in `errors/00.npy` through `errors/09.npy` in table order. The two error measures include all *BCHW* elements:

$$E_{\max} = \max_{b,c,h,w} |I[b, c, h, w] - \hat{I}[b, c, h, w]|,$$

$$\text{MAE} = \frac{1}{BCHW} \sum_{b,c,h,w} |I[b, c, h, w] - \hat{I}[b, c, h, w]|.$$

4. Results (measured values from results.csv)

Input	<i>B</i>	<i>C</i>	<i>H</i>	<i>W</i>	$E_{\max}$	MAE
Manual ordering	1	2	2	3	0.0	0.0
MNIST test samples 0-1	2	1	28	28	0.0	0.0
Synthetic RGB	2	3	32	32	0.0	0.0
Synthetic fmap C=8	2	8	13	17	0.0	0.0
Synthetic fmap C=16	2	16	13	17	0.0	0.0
Synthetic fmap C=32	2	32	13	17	0.0	0.0
Synthetic fmap C=64	2	64	13	17	0.0	0.0
Synthetic fmap C=128	2	128	13	17	0.0	0.0
Synthetic fmap C=256	2	256	13	17	0.0	0.0
Synthetic fmap C=500	2	500	13	17	0.0	0.0

10 / 10 experiments passed; 451,492 elements verified.

MNIST. The first two test images (indices 0 and 1) are loaded from the original IDX archive as unnormalized `uint8` grayscale values. The archive is included in `data/`; its SHA-256 checksum and IDX header are validated before use.

Synthetic data. The three-channel input is synthetic RGB-shaped data, not CIFAR. Higher-channel inputs are synthetic `float32` feature maps, not outputs of a trained network. Each case uses a fresh NumPy PCG64 generator with seed $20261005 + C$ and a standard normal distribution. Dimensions remain practical: the largest experiment contains 221,000 elements.

All measured maximum absolute errors and MAEs are 0.0, and every saved signed difference is zero. Neither conversion performs arithmetic on tensor values; both preserve dtype, so exact reconstruction is expected.

Results CSV SHA-256:

6d20234babae59337033c2b75f0d99b5e20c15391c590660cfaad1c3c5f8b634

Validation and discussion

5. Verification

The 16 automated unit tests cover handwritten expected ordering, $B > 1$, non-square height/width, singleton axes, all nine requested channel counts, four numeric dtypes, noncontiguous inputs, index bijection, input preservation, invalid inputs, and explicit-loop implementation. Deliberate matrix permutations and reconstruction corruption must be detected.

An independent test uses unsigned inputs with expected signed differences $[-255, 255, 2, -3]$, maximum absolute error 255, and MAE 128.75. This verifies the error calculator and guards against unsigned subtraction wraparound. Integer values are subtracted before conversion to floating-point metrics; a separate test checks differences between large `int64` values.

The conversion call audit allows only validation, allocation, loops and error construction. No `reshape`, `view`, `flatten`, `ravel`, or equivalent library layout operation is used in either algorithm. No optional library layout-conversion reference is used.

6. Observations and limitations

Zero error from $C = 1$ through $C = 500$ supports correct channel boundaries and preservation of the batch dimension. An independent ordering check is necessary: two mutually consistent but incorrect permutations could pass a round trip. Quotient/remainder tests establish that each valid linear address has exactly one BCHW coordinate.

The implementation uses Python loops to explain address generation, not to benchmark accelerator throughput. It supports positive dimensions and finite real integer/floating-point NumPy arrays. Empty axes, NaN, infinity, complex and object values are rejected. Differences and aggregate metrics use `float64` storage; exact integer subtraction precedes storage. Arbitrary precision error reporting for extreme values is outside these experiments.

7. Reproducibility and sources

The measured experiments used Python 3.14.2 and NumPy 2.4.2. Run the unit tests and `experiments.py` to reproduce the results.

Next, `build_report.py` refreshes measured content in this editable source and invokes LaTeX to compile its PDF. Run `verify_submission.py` to compare the LaTeX/PDF tables with the CSV, console, manifest and saved differences. Setup commands, data provenance and the requirement checklist are in `EVALUATOR.md`.

Assignment source: user-provided `assgn3-tensor-flat.pdf`, pages 1–3.

Dataset source: CVDF’s public mirror of the MNIST test images.

<https://github.com/cvdfoundation/mnist>

The archive was downloaded on 5 October 2026 and included unchanged for offline experiment reproduction. The RGB-shaped and higher-channel cases are explicitly synthetic.

Editing: this standalone `report.tex` can be compiled in Overleaf, with pdfLaTeX/XeLaTeX, or with Tectonic. Edit the name/roll macros near the top. The generated blocks preserve the measured results; other prose is freely editable.